
CSC411 - Artificial Intelligence | Semester Project Report

THE AI SENTIENCE HUB

A 3-in-1 Arcade for AI Algorithms

MOD-01 EvoMap

Genetic Algorithm + BFS Dungeon Generator

MOD-02 Aegis Grid

A* Pathfinding vs Minimax Adversary

MOD-03 Crypto-Glyph

Handwritten Digit Recognition — NB, DT, KNN

Submitted by: Anood Tayyeba Imtiaz (01-135232-010)
Muhammad Tayyaab Zahoor (01-135232-070)

Class: BSIT 6B

Instructor: Ms. Sidra Ayesha

Course: CSC411 — Artificial Intelligence

Date: May 2026

Built with Python3 · pygame-ce · scikit-learn · NumPy · scipy

Contents

1	Introduction	3
2	Problem Statement	3
3	Methodology / Algorithms Used	4
3.1	Uninformed Search — Breadth-First Search (BFS)	4
3.2	Informed Search — A* Algorithm	5
3.3	Adversarial Search — Minimax with Alpha-Beta Pruning	7
3.4	Evolutionary Computation — Genetic Algorithm	8
3.5	Machine Learning — Supervised Classification	10
3.5.1	Preprocessing	10
3.5.2	Naive Bayes	11
3.5.3	Decision Tree	11
3.5.4	K-Nearest Neighbours	11
3.5.5	Online / Incremental Learning	12
4	Flowchart / System Design Diagram	13
4.1	Top-Level System Architecture	13
4.2	EvoMap — Genetic Algorithm Flowchart	14
4.3	Aegis Grid — A* vs Minimax Flowchart	15
4.4	Crypto-Glyph — Classification Pipeline Flowchart	16
5	Step-by-Step Module Working	16
5.1	MOD-01: EvoMap	16
5.2	MOD-02: Aegis Grid	17
5.3	MOD-03: Crypto-Glyph	17
6	Implementation Details	18
6.1	Project Structure	18
6.2	Technology Stack	19
6.3	Key Implementation Decisions	19
7	Complete Code	20
8	Results and Output Screenshots	20
8.1	Main Hub — Application Entry Point	20
8.2	EvoMap — Dungeon Evolution in Progress	21
8.3	Aegis Grid — A* Visualisation Mode	21
8.4	Aegis Grid — Adversarial Mode	22

8.5	Crypto-Glyph — Digit Recognition	22
8.6	Model Accuracy Results	22
8.7	Per-Digit Performance (KNN — Best Model)	23
8.8	Application Usage Statistics	24
8.9	Observed Behaviour	24
9	Challenges and Solutions	25
10	Individual Contribution of Each Group Member	26
11	Conclusion	28

1. Introduction

Most people who study Artificial Intelligence in a classroom end up staring at numbers on a black terminal screen. The algorithms are running, the numbers are changing, but you never really *see* the AI think. You cannot watch BFS expand outward like a flood filling a space, you cannot see a dungeon grow better over 60 generations of evolution, and you cannot watch three classifiers argue about whether your scribble is a 3 or an 8.

That is the problem this project was built to solve.

The AI Sentience Hub is a complete desktop application written in Python that acts like a video game main menu. Instead of one isolated script, it is a unified graphical hub containing three highly visual and interactive programs. Each one demonstrates a different group of AI concepts from CSC411 in real time, with things you can actually click on and play with.

The three modules are:

- **EvoMap** — A dungeon generator that uses a Genetic Algorithm to evolve 50×50 tile maps over 60 generations, with BFS verifying that a walkable path from the Spawn tile to the Boss tile exists at every step of the way.
- **Aegis Grid** — A grid game where a Red agent finds the shortest path to the goal using A* Search, while a Blue agent tries to block it by placing walls. Blue's decisions are driven by the Minimax algorithm with Alpha-Beta Pruning.
- **Crypto-Glyph** — A drawing pad where you draw any digit from 0–9 and three trained Machine Learning classifiers (Naive Bayes, Decision Tree, and KNN) instantly vote on what you wrote, then update their own models in real time based on your feedback.

All three modules run inside a polished main hub menu with animated backgrounds, a live usage counter, and info modals that explain the AI theory behind each module. Everything is written in Python using `pygame-ce` for graphics and `scikit-learn` for the machine learning components.

2. Problem Statement

AI algorithms are abstract. A student can memorize the steps of A* or understand Minimax on paper, but without seeing these algorithms operate in a real and interactive environment, the understanding stays shallow and fades quickly after the exam.

The following gaps exist in how AI is typically taught at the undergraduate level:

- There is no intuitive way to *see* BFS expanding through a state space and understand why it guarantees the shortest path by exploring level by level.
- Minimax and Alpha-Beta Pruning are hard to fully grasp without watching a real adversary make decisions and seeing which branches get pruned in real time.
- Genetic Algorithms are often taught as a dry list of steps (select, crossover, mutate) without showing how fitness actually improves across generations, or what a generation 1 dungeon looks like compared to a generation 60 dungeon.
- ML classifiers are typically demonstrated through accuracy numbers on a report, without giving students a way to interact with the model, draw their own inputs, see where the model fails, or teach it new examples.

The goal of this project is to build an interactive system that makes all of these concepts visible, playable, and genuinely understandable — covering uninformed search, informed search, adversarial search, evolutionary computation, and supervised machine learning in a single unified application.

3. Methodology / Algorithms Used

3.1. Uninformed Search — Breadth-First Search (BFS)

BFS is the backbone of EvoMap. Every dungeon generated by the Genetic Algorithm is checked by BFS to confirm that a valid traversable path exists from the Spawn tile to the Boss tile. If no such path exists, the dungeon receives a fitness score of zero and is immediately eliminated from the population.

BFS works by exploring all nodes at distance 1 from the start before moving on to distance 2, then distance 3, and so on. Because it never skips a level, it always finds the *shortest* possible path. In our case, the dungeon is a 50×50 grid where each walkable cell is a node and adjacency (up, down, left, right) defines edges.

We implement three BFS variants: `_bfs()` which returns all reachable cells from a starting point, `_bfs_path_length()` which returns the exact step count to a target, and `get_path_through()` which chains multiple BFS segments together to find a path through a sequence of waypoints (Spawn → Treasure → Boss).

```
1 def _bfs_path_length(dungeon, start, target):
2     """BFS from start to target. Returns step count, or None if
3         unreachable."""
4     visited = set()
5     visited.add(start)
6     queue = deque([(start, 0)])
```

```

6
7     while queue:
8         (r, c), dist = queue.popleft()
9         for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
10            nr, nc = r + dr, c + dc
11            if 0 <= nr < GRID_H and 0 <= nc < GRID_W:
12                if dungeon[nr, nc] != WALL and (nr, nc) not in
                    visited:
13                    if (nr, nc) == target:
14                        return dist + 1
15                    visited.add((nr, nc))
16                    queue.append((nr, nc), dist + 1)
17     return None # unreachable

```

Listing 1: BFS shortest path check — evomap.py

Time complexity: $O(W \times H)$ per dungeon evaluation, where $W = H = 50$.

BFS also appears in Aegis Grid, inside the `generate_maze()` function. After the 20×20 maze is procedurally built, BFS runs from the start cell (1,1) to the goal (18,18) to confirm the maze is actually solvable before any game begins. If the path check fails, the maze is replaced with a fully open grid, guaranteeing the player is never given an impossible board.

3.2. Informed Search — A* Algorithm

A* drives the Red agent in Aegis Grid. Unlike BFS which treats all steps as equally costly, A* uses a heuristic function to estimate how far the goal is from any given node. This lets it prioritize more promising directions and find the shortest path much faster than BFS on large grids.

The evaluation function is:

$$f(n) = g(n) + h(n)$$

where $g(n)$ is the exact cost from the start to node n (number of steps taken), and $h(n)$ is the heuristic estimate of the remaining distance from n to the goal.

We implemented three interchangeable heuristics, selectable by the user at any time during the game:

Heuristic	Formula	Note
Manhattan	$ \Delta r + \Delta c $	Correct for 4-directional movement
Euclidean	$\sqrt{\Delta r^2 + \Delta c^2}$	Slightly underestimates on grid — explores more nodes
Chebyshev	$\max(\Delta r , \Delta c)$	Designed for 8-directional movement

A priority queue (`heapq`) keeps the open set sorted by $f(n)$. We maintain a separate `open_set_tracker` set for $O(1)$ membership checks, which avoids a known performance pitfall where stale entries in the heap cause redundant processing.

```

1 def a_star(grid, start, goal, h_type):
2     open_set = []
3     heapq.heappush(open_set, (0, 0, start))
4     open_set_tracker = {start}
5     came_from = {}
6     g_score = {start: 0}
7
8     while open_set:
9         _, _, current = heapq.heappop(open_set)
10        if current not in open_set_tracker:
11            continue          # stale heap entry -- skip
12        open_set_tracker.remove(current)
13
14        if current == goal:
15            p = []
16            while current in came_from:
17                p.append(current)
18                current = came_from[current]
19            p.append(start)
20            return list(reversed(p)), open_history, closed_history,
                nodes_explored
21
22        for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
23            nbr = (current[0] + dr, current[1] + dc)
24            if 0 <= nbr[0] < 20 and 0 <= nbr[1] < 20 and grid[nbr] !=
                1:
25                tg = g_score[current] + 1
26                if tg < g_score.get(nbr, float('inf')):
27                    came_from[nbr] = current
28                    g_score[nbr] = tg
29                    f = tg + get_heuristic(nbr, goal, h_type)
30                    heapq.heappush(open_set, (f, tg, nbr))
31                    open_set_tracker.add(nbr)
32    return [], open_history, closed_history, nodes_explored    # no
    path found

```

Listing 2: A* implementation – `aegis_grid.py`

Time complexity: $O(E \log V)$, where E is the number of edges (walkable adjacencies) and V is the number of reachable nodes.

3.3. Adversarial Search — Minimax with Alpha-Beta Pruning

The Blue agent in Aegis Grid uses Minimax to decide where to place a wall. Minimax works by building a game tree of future states. It assumes Red will always play optimally to minimize its own path length, and Blue will always play optimally to maximize it. By looking ahead several moves, Blue can identify the wall placement that causes the most damage to Red's route.

The recursive definition is:

$$\text{Minimax}(n, d) = \begin{cases} \text{len}(A^*\text{path}) & \text{if } d = 0 \text{ or terminal state} \\ \max_a \text{Minimax}(\text{result}(n, a), d-1) & \text{if Blue's turn (maximize)} \\ \min_a \text{Minimax}(\text{result}(n, a), d-1) & \text{if Red's turn (minimize)} \end{cases}$$

We search to a depth of 3, meaning Blue considers walls, Red's response, and Blue's follow-up. Alpha-Beta Pruning is layered on top: it maintains a running α (the best score Blue has found so far) and β (the best score Red has found so far). When $\beta \leq \alpha$, the current branch cannot possibly affect the final decision, so it is pruned entirely.

```

1  # Blue's turn (maximising Red's path length)
2  for r, c in candidate_walls:
3      g[r, c] = 1                # temporarily place wall
4      v, _ = _mm(g, r_pos, d-1, alpha, beta, is_blue=False)
5      g[r, c] = 0                # undo wall (search is non-destructive)
6      if v > best_val:
7          best_val = v
8          best_action = (r, c)
9      alpha = max(alpha, best_val)
10     if beta <= alpha:           # Red would never allow this branch
11         pruned_count += 1
12         break

```

Listing 3: Alpha-Beta Pruning inside `_mm()` — `aegis_grid.py`

Candidate pruning. Rather than evaluating all 400 possible empty cells on the 20×20 grid, Blue only considers cells within Manhattan distance 2 of Red's current A* path, capped at 15 candidates sorted by proximity to the path's midpoint. This means Blue focuses on strategically relevant positions — the middle of Red's route — rather than wasting computation on corner cells that would not change anything.

Complexity: $O(b^{d/2})$ average case with Alpha-Beta Pruning, compared to $O(b^d)$ without it, where b is the branching factor and $d = 3$ is the search depth.

3.4. Evolutionary Computation — Genetic Algorithm

EvoMap uses a Genetic Algorithm (GA) to evolve a population of 30 dungeons over 60 generations. Each generation, the weakest dungeons are discarded and the best ones breed to produce new offspring. Over time, the dungeons get better — more traversable, more complex, and structurally richer.

The full GA pipeline:

- **Initialization:** 30 random dungeons are created. Each dungeon is a 50×50 NumPy grid. Rooms (4–10 tiles wide) are placed without overlap, then connected by L-shaped corridors. Spawn, Boss, and Treasure tiles are placed at the centres of specific rooms.
- **Fitness Evaluation:** Every dungeon is scored by the fitness function (see below). Dungeons with no path from Spawn to Boss score exactly 0.
- **Selection (Elitism):** The top 50% by fitness survive unchanged into the next generation. This guarantees the best solution never gets worse.
- **Crossover:** Two survivors are selected at random. A horizontal band of rows (random start and end) is copied from one parent and placed into the other, creating a child dungeon that inherits structure from both parents.
- **Mutation:** Random tiles are flipped between Wall and Floor at a 5% rate. Special tiles (Spawn, Boss, Treasure) are protected from mutation.
- **Repair:** After crossover and mutation, the offspring is scanned. If any special tile is missing, it is placed back on the nearest available floor cell. This ensures 100% of the population is structurally valid before fitness is evaluated.

```
1 def _crossover(parent_a, parent_b):
2     """Row-splice crossover: copy a random horizontal band from
3         parent_b."""
4     child = parent_a.copy()
5     cut1 = random.randint(0, GRID_H - 2)
6     cut2 = random.randint(cut1 + 1, GRID_H)
7     child[cut1:cut2, :] = parent_b[cut1:cut2, :]
8     return child
```

Listing 4: Crossover operator — evomap.py

```
1 def _mutate(dungeon):
2     """Flip wall <-> floor at MUTATION_RATE probability per tile."""
3     mutated = dungeon.copy()
4     mask = np.random.random((GRID_H, GRID_W)) < MUTATION_RATE
```

```

5     special = np.isin(mutated, [SPAWN, BOSS, TREASURE])    # protect
      special tiles
6     mutated[mask & ~special & (mutated == WALL)] = FLOOR
7     mutated[mask & ~special & (mutated != WALL)] = WALL
8     return mutated

```

Listing 5: Mutation operator — evomap.py

```

1 def _repair(dungeon):
2     """Ensure Spawn, Boss, and Treasure always exist after genetic
      operations."""
3     repaired = dungeon.copy()
4     for tile, fallback_fn in [(SPAWN, lambda c: c[0]),
5                               (BOSS, lambda c: c[-1]),
6                               (TREASURE, lambda c: c[len(c)//2])]:
7         if _find_tile(repaired, tile) is None:
8             candidates = list(map(tuple, np.argwhere(repaired ==
9               FLOOR)))
10            if candidates:
11                r, c = fallback_fn(candidates)
12                repaired[r, c] = tile
13    return repaired

```

Listing 6: Repair operator — evomap.py

The Fitness Function is the most important piece. It rewards dungeons that are fully traversable, structurally complex, and have a Treasure that is worth collecting:

$$\text{fitness} = \underbrace{\frac{|\text{reachable}|}{|\text{walkable}|}}_{\text{reachability}} + 0.1 \times \text{rooms} - 0.05 \times \text{dead_ends} + \max(0, 0.5 - \text{detour} \times 0.01)$$

The **detour** term is particularly interesting. It calculates how many extra steps a player must walk to collect the Treasure compared to going straight to the Boss. If $\text{detour} = 0$, the Treasure lies exactly on the optimal path and the full bonus of $+0.5$ is awarded. If the Treasure is buried in a dead-end far from the main route, the bonus shrinks toward zero.

Room counting uses `scipy.ndimage.label` (connected component labelling on the floor mask), with a BFS fallback if `scipy` is unavailable. This counts distinct room regions, not just individual tiles.

```

1 def fitness_score(dungeon):
2     spawn_pos = _find_tile(dungeon, SPAWN)
3     boss_pos  = _find_tile(dungeon, BOSS)

```

```

4     if spawn_pos is None or boss_pos is None:
5         return 0.0
6
7     reachable = _bfs(dungeon, spawn_pos)
8     if boss_pos not in reachable:
9         return 0.0          # no path to boss: instant zero, dungeon
                             # is eliminated
10
11     total_walkable = int(np.count_nonzero(dungeon))
12     reachability   = len(reachable) / total_walkable
13     room_count     = _count_rooms(dungeon)      # scipy connected
                             # components
14     dead_ends      = _count_dead_ends(dungeon)
15     score          = reachability + room_count * 0.1 - dead_ends *
                             0.05
16
17     # Treasure detour scoring
18     s2t = _bfs_path_length(dungeon, spawn_pos, treasure_pos)
19     t2b = _bfs_path_length(dungeon, treasure_pos, boss_pos)
20     s2b = _bfs_path_length(dungeon, spawn_pos, boss_pos)
21     if s2t and t2b and s2b:
22         detour = (s2t + t2b) - s2b
23         score += max(0.0, 0.5 - detour * 0.01)
24
25     return max(score, 0.0)

```

Listing 7: Fitness function — evomap.py

3.5. Machine Learning — Supervised Classification

Crypto-Glyph trains and deploys three classifiers on the MNIST dataset: 70,000 hand-written digit images, each 28×28 pixels, giving 784 input features per sample.

3.5.1. Preprocessing

Before any user drawing is classified, it goes through a preprocessing pipeline designed to match the MNIST format exactly:

1. The canvas is cropped to the bounding box of the drawn pixels, removing empty borders.
2. The cropped image is scaled so its largest dimension is 20 pixels, then centered on a 28×28 black canvas. This matches how MNIST digits are formatted (20×20 content, 4-pixel border).
3. A light Gaussian blur (radius 0.5) is applied to smooth jagged mouse strokes.

4. The 784 pixel values (0–255) are flattened to a 1D vector and normalized using a pre-fitted `StandardScaler` to zero mean and unit variance. Without this step, KNN’s distance calculations would be dominated by high-intensity pixels.

3.5.2. Naive Bayes

Gaussian Naive Bayes applies Bayes’ theorem under the assumption that each pixel feature is conditionally independent given the class:

$$P(\text{class} \mid \mathbf{x}) \propto P(\text{class}) \cdot \prod_{i=1}^{784} P(x_i \mid \text{class})$$

It trains in under 1 second and supports online learning through `partial_fit()`, which updates the model’s Gaussian parameters with a single new sample without retraining from scratch. The trade-off is accuracy: the independence assumption is fundamentally wrong for image data, where neighbouring pixels are strongly correlated. This is why Naive Bayes achieves only 52.4% accuracy. That weakness is itself educational — it shows exactly what goes wrong when the wrong model is applied to the wrong type of data.

3.5.3. Decision Tree

The Decision Tree recursively partitions the 784-dimensional feature space. At each node, it finds the pixel threshold that minimizes Gini impurity:

$$\text{Gini}(t) = 1 - \sum_{k=0}^9 p_k^2$$

where p_k is the proportion of class k samples at node t . It is capped at `max_depth=20` to prevent overfitting. Deeper trees would memorize the training data and generalize poorly to user drawings. It achieves 88.2% accuracy and is retrained in batches of 5 user-feedback corrections.

3.5.4. K-Nearest Neighbours

KNN stores the entire training set and classifies a new drawing by finding the 5 closest training examples in 784-dimensional Euclidean space, then taking a majority vote:

$$\hat{y} = \text{majority_vote}(\{y_i : i \in \text{kNN}(\mathbf{x})\})$$

It makes no distributional assumptions about the data, which is why it outperforms both

other models at 92.8% accuracy. We use `algorithm="ball_tree"`, a space-partitioning data structure that reduces nearest-neighbour search from $O(n \times d)$ brute-force to approximately $O(d \times \log n)$, where $d = 784$ and n is the training set size.

KNN was trained on 20,000 of the 60,000 MNIST training samples. This keeps the model file at a manageable 97MB while maintaining competitive accuracy. The full 60,000 sample model exceeded 200 MB and was too slow to load at startup.

3.5.5. Online / Incremental Learning

After every prediction, the user can tell the system which model was right and what the digit actually was. This triggers a background retraining worker that updates all three models without restarting from scratch:

```

1 def _retrain_worker(true_label, raw_arr, scaled_arr):
2     # Naive Bayes: single-sample online update via Bayes parameter
      update
3     nb_model.partial_fit(scaled_arr, [true_label], classes=np.arange
      (10))
4     joblib.dump(nb_model, "models/naive_bayes.pkl")
5
6     # Decision Tree: batch retrain after accumulating 5 corrections
7     dt_feedback_buffer.append((scaled_arr, true_label))
8     if len(dt_feedback_buffer) >= 5:
9         X = np.vstack([x for x, _ in dt_feedback_buffer])
10        y = np.array([l for _, l in dt_feedback_buffer])
11        dt_model.fit(X, y)
12        joblib.dump(dt_model, "models/decision_tree.pkl")
13
14    # KNN: grow the training set and refit
15    X_base, y_base = joblib.load("models/knn_train_data.pkl")
16    X_combined = np.vstack([X_base, scaler.transform(np.vstack(
      knn_extra_X))])
17    y_combined = np.concatenate([y_base, np.array(knn_extra_y)])
18    knn_model.fit(X_combined, y_combined)
19    joblib.dump(knn_model, "models/knn.pkl")
20
21    with model_lock:
22        retraining_in_progress = False # release the flag inside the
      mutex

```

Listing 8: Incremental retraining worker — `crypto_glyph.py`

The retraining runs on a daemon thread. A `threading.Lock()` (mutex) protects all model read and write operations so the UI thread never reads a model that is halfway through being updated.

4. Flowchart / System Design Diagram

4.1. Top-Level System Architecture

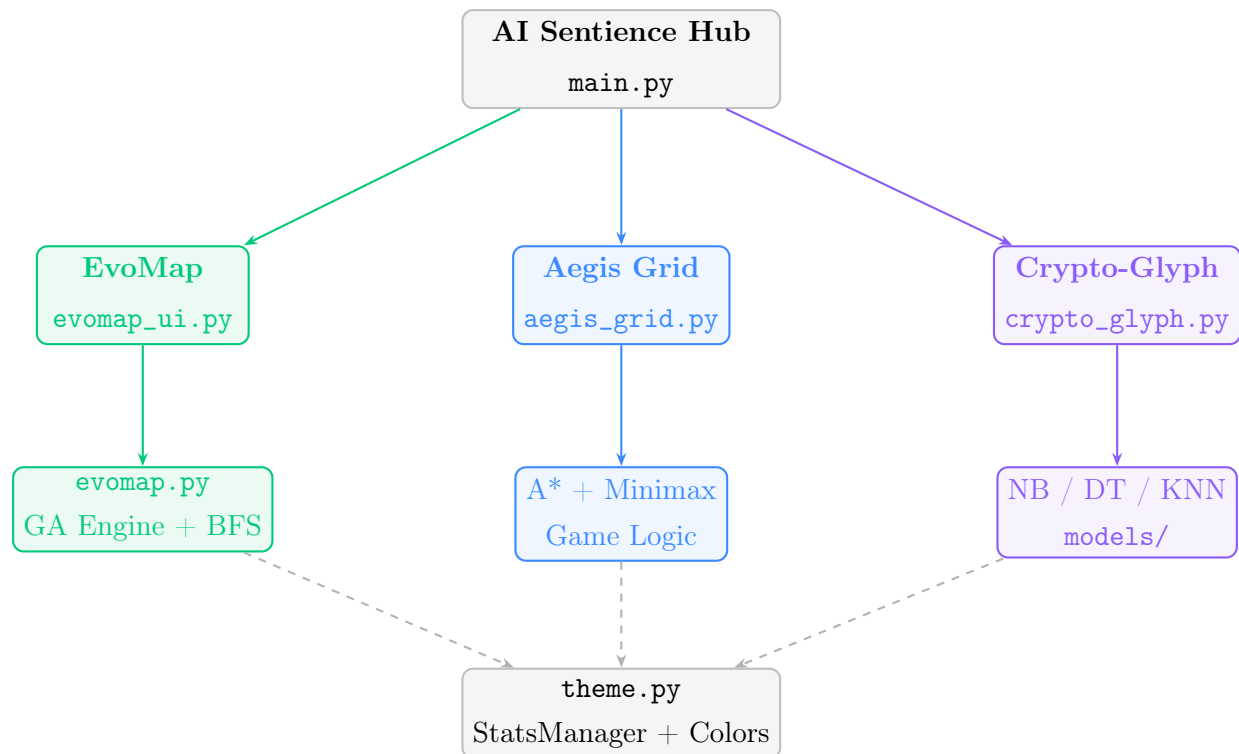


Figure 1: Top-level system architecture. Each module is launched as a subprocess from the main hub and returns control when closed. All modules share `theme.py` and write usage counts to `stats.json` via `StatsManager`.

4.2. EvoMap — Genetic Algorithm Flowchart

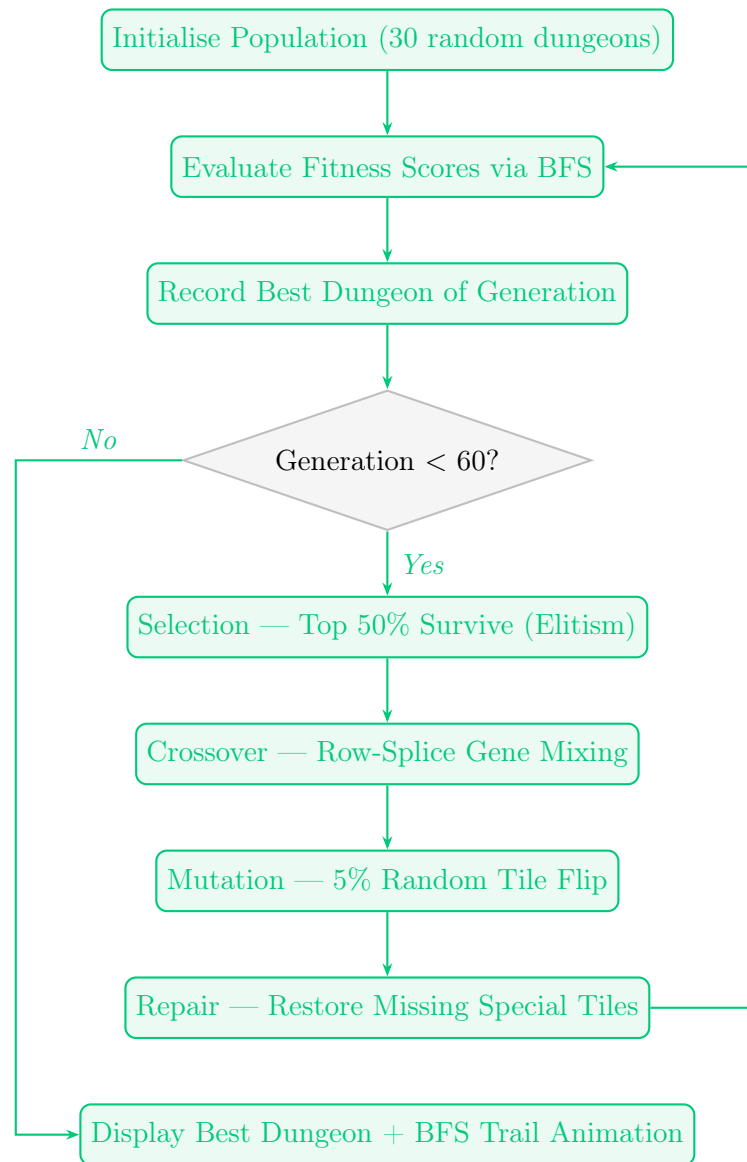


Figure 2: EvoMap — full Genetic Algorithm pipeline. The loop runs on a background daemon thread so the UI stays responsive throughout all 60 generations.

4.3. Aegis Grid — A* vs Minimax Flowchart

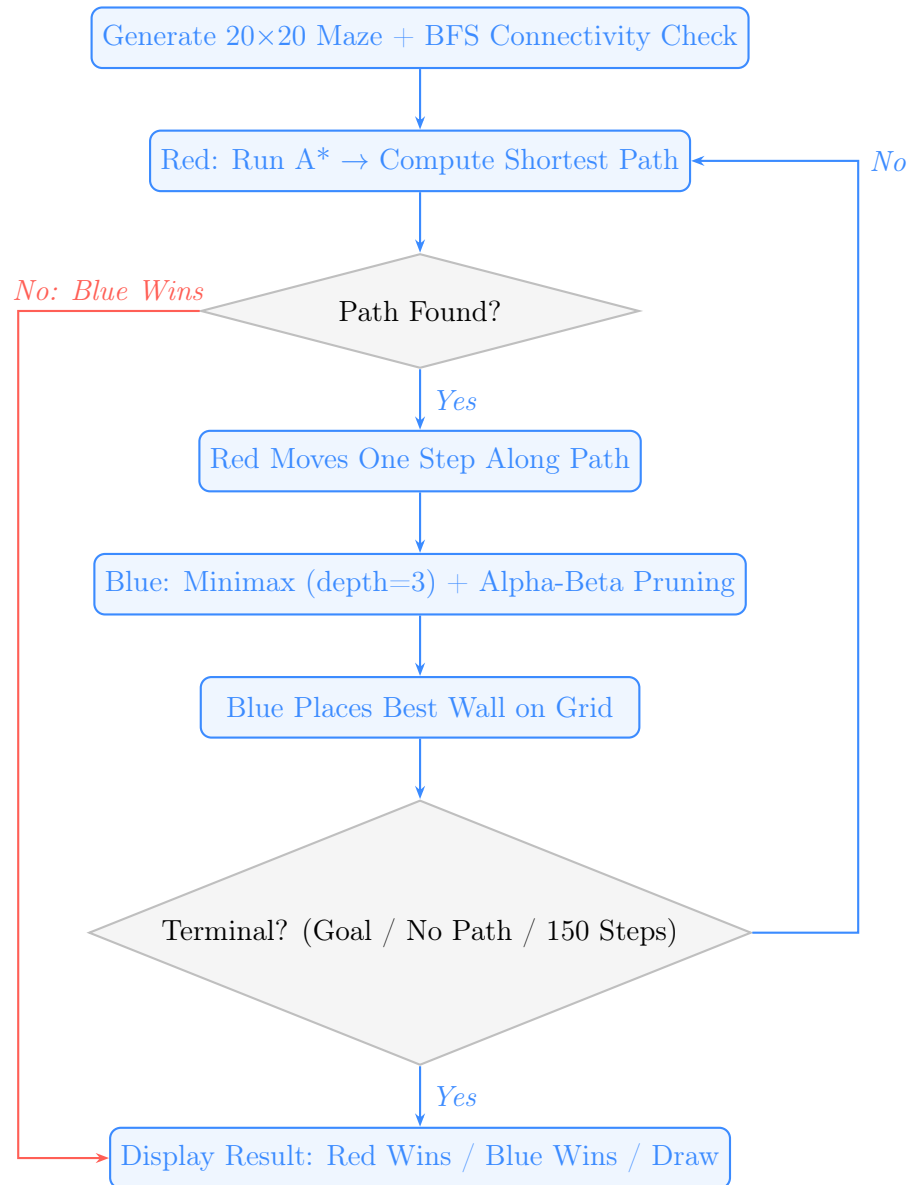


Figure 3: Aegis Grid — game loop. Both agents run on background threads to keep the UI at 60 FPS. An “is_thinking” flag prevents overlapping computation.

4.4. Crypto-Glyph — Classification Pipeline Flowchart

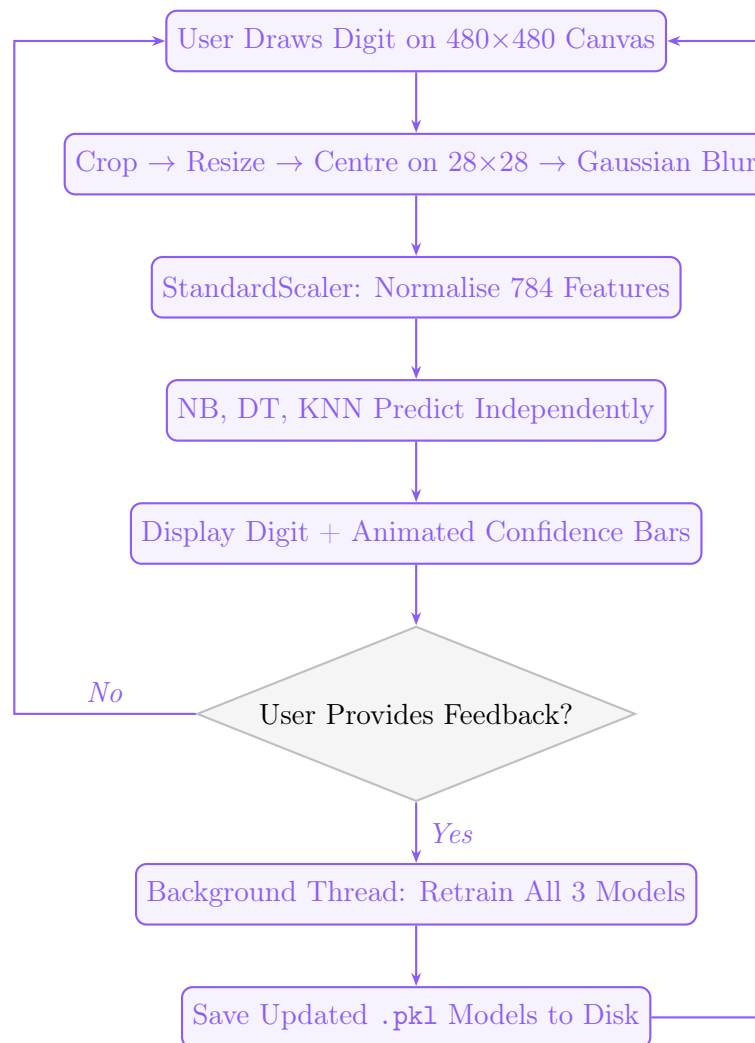


Figure 4: Crypto-Glyph — classification and online learning pipeline. Retraining runs on a daemon thread protected by a mutex so the UI never freezes.

5. Step-by-Step Module Working

5.1. MOD-01: EvoMap

Step 1 — Configure sliders. Three sliders at the bottom of the EvoMap window let the user set the Mutation Rate (0–20%), Population Size (10–100), and Target Room Count (5–15). These values are passed directly into the GA before it runs.

Step 2 — Evolution runs on a background thread. Pressing “RUN EVOLUTION” starts a Python daemon thread that runs the full `evolve()` pipeline. A shared `evolution_progress` integer is incremented by a callback after each generation; the main thread reads this every frame to animate a smooth progress bar without freezing.

Step 3 — Generation playback. Once all 60 generations complete, the UI plays them

back frame by frame. A fitness line chart draws generation by generation in real time, letting you see exactly when the population crosses from zero-scoring dungeons to valid ones.

Step 4 — BFS trail animation. After the final best dungeon is shown, an animated cyan dot marches from the Spawn tile to the Boss tile (routing through the Treasure if it sits on the optimal path). This proves visually that the dungeon is solvable.

Step 5 — Dashboard metrics. The right panel displays Best Fitness Score, BFS Path Length, Room Count, Steps to Treasure, and the Treasure Detour Bonus — all updating live as the user scrubs through generations with the arrow keys.

5.2. MOD-02: Aegis Grid

The module has two operational modes, switchable at any time.

Visualise Mode runs A* on the current grid and animates the search step by step. Orange cells show the Open Set (nodes queued for exploration) and grey cells show the Closed Set (already evaluated). The user can click any cell to add or remove a wall and watch A* recalculate from scratch instantly. Three heuristic buttons (Manhattan, Euclidean, Chebyshev) let the user compare how each one affects the number of nodes explored.

Adversarial Mode runs the full game. Red moves one step per game tick using A*. After each Red step, Blue's Minimax evaluates up to 15 candidate wall positions, looks 3 moves ahead, and places the wall that most extends Red's path. The dashboard shows Nodes Explored, Current Path Length, Alpha-Beta Branches Pruned, and a Session Win counter. A **Human vs AI mode** is also available, where the user manually places walls as Blue to test their own strategy against A*.

5.3. MOD-03: Crypto-Glyph

Step 1 — Drawing. The user draws on a 480×480 white canvas with three brush sizes (S, M, L), an eraser, and stroke-by-stroke undo.

Step 2 — Prediction. On mouse release, the drawing is preprocessed and fed to all three classifiers in sequence. Each returns a digit label and a full probability distribution over digits 0–9.

Step 3 — Confidence display. Animated bar charts show each model's confidence for every digit class. The bars animate smoothly from their previous position to the new probabilities. The winning prediction is highlighted in the model's accent color.

Step 4 — Feedback. Four buttons appear below the results: “NB ✓”, “DT ✓”,

“KNN ✓”, and “NONE”. The user selects which model was correct. If none was right, they can type the actual digit and confirm. This triggers background retraining.

Step 5 — Confusion Matrix Viewer. A modal shows a full 10×10 confusion matrix for any of the three classifiers, color-coded by row-normalised intensity. Diagonal cells (correct predictions) are marked with a white outline. This makes it easy to see which digit pairs cause the most confusion.

6. Implementation Details

6.1. Project Structure

```
project/
|---- main.py           Hub menu, animated background, card routing
|---- evomap.py         GA engine, fitness, BFS, crossover, repair
|---- evomap_ui.py      EvoMap UI: sliders, playback, BFS animation
|---- aegis_grid.py     A*, Minimax, maze generation, full game loop
|---- crypto_glyph.py   Drawing pad, ML inference, online retraining
|---- theme.py          Shared colors, fonts, StatsManager
|---- train_models.py   Offline MNIST training script (run once)
|---- requirements.txt  Python dependencies
|---- stats.json        Persistent usage counters
models/
|---- naive_bayes.pkl   Trained GaussianNB (0.57s train time)
|---- decision_tree.pkl Trained DecisionTreeClassifier depth=20
|---- knn.pkl           Trained KNeighborsClassifier k=5 (97 MB)
|---- scaler.pkl        Fitted StandardScaler
|---- knn_train_data.pkl Base training set for KNN online updates
|---- accuracy.json     Per-model benchmark metrics
\---- feedback.pkl      Persisted user correction history
```

6.2. Technology Stack

Library	Purpose
<code>pygame-ce</code>	All graphics, event handling, animation, and UI rendering
<code>scikit-learn</code>	GaussianNB, DecisionTreeClassifier, KNeighborsClassifier, StandardScaler, <code>confusion_matrix</code>
<code>numpy</code>	50×50 dungeon grids, image arrays, vectorised operations
<code>scipy.ndimage</code>	Connected-component labelling for counting dungeon rooms
<code>joblib</code>	Model serialisation and deserialisation (.pkl files)
<code>Pillow</code>	Image preprocessing: crop, resize, Gaussian blur
<code>threading</code>	Background GA evolution and model retraining workers
<code>heapq</code>	Priority queue (min-heap) for A* open set
<code>psutil</code>	Live CPU and memory usage displayed in the hub status bar

6.3. Key Implementation Decisions

Threading for heavy computation. Both the Genetic Algorithm (EvoMap) and the model retraining (Crypto-Glyph) run on Python daemon threads. The main Pygame event loop runs at 60 FPS and never blocks. A shared progress integer and a `threading.Lock()` handle all communication between threads safely.

Surface caching for rendering performance. The 50×50 dungeon tile map is pre-rendered to a `pygame.Surface` once per generation and stored in a dictionary keyed by generation index. On each frame, only the animated overlay (trail dots, flash effects) is drawn on top of the cached surface. Without this, BFS traversals and full grid redraws would run on every frame.

Repair operator enforces hard constraints. Row-splice crossover frequently overwrites Spawn, Boss, or Treasure tiles. Rather than penalising these in the fitness function (which wastes evaluation time), a repair pass runs immediately after crossover and mutation. This ensures 100% of the population entering fitness evaluation is structurally valid.

KNN ball-tree algorithm. `algorithm="ball_tree"` is passed to KNN in training. Without it, scikit-learn defaults to brute-force nearest neighbour search — $O(n \times d)$ per query — which is unacceptably slow at 784 dimensions. Ball-tree reduces this to approximately $O(d \times \log n)$.

Minimax candidate pruning. Evaluating all 400 possible wall cells at depth 3 makes the game unplayable. Blue instead evaluates only cells within Manhattan distance 2 of Red's current A* path, capped at 15 candidates. This preserves strategically relevant

blocking behaviour while keeping Blue's turn under 0.5 seconds.

7. Complete Code

The complete source code is submitted as a separate archive (`Project.zip`). Pre-trained model files are included in the `models/` subdirectory. The key files and their sizes are listed below.

File	Lines	Responsibility
<code>aegis_grid.py</code>	787	A*, Minimax, maze generation, full game UI
<code>crypto_glyph.py</code>	705	Drawing pad, classification, online retraining
<code>evomap_ui.py</code>	816	GA playback, sliders, BFS trail animation
<code>main.py</code>	549	Hub menu, animated cards, subprocess routing
<code>evomap.py</code>	498	GA engine, fitness function, BFS logic
<code>theme.py</code>	251	Colors, fonts, StatsManager, shared widgets
<code>train_models.py</code>	92	Offline MNIST training and evaluation script

To run the project on any machine:

```
1 pip install pygame-ce numpy scikit-learn scipy pillow joblib psutil
2 # Place the models/ folder in the same directory as the source files
3 python main.py
```

Listing 9: Setup and execution

8. Results and Output Screenshots

8.1. Main Hub — Application Entry Point

Main Hub - `main.py`

The main hub displays three module cards (EvoMap in green, Aegis Grid in blue, Crypto-Glyph in purple) on a dark animated background. Fifty floating nodes connected by proximity-based edges drift across the screen. A stats bar at the bottom shows live dungeon/game/digit counts. Each card has a live animated thumbnail preview and an info button that opens a modal explaining the AI theory.

Figure 5: The AI Sentience Hub main menu. The system status bar shows live CPU%, RAM usage, and how many ML models are loaded.

8.2. EvoMap — Dungeon Evolution in Progress

EvoMap - evomap_ui.py

Left side: the 50×50 dungeon grid with colour-coded tiles (dark=wall, green tint=floor, bright green=spawn, red=boss, gold=treasure). Right side dashboard: generation counter, best fitness score, a live line chart of fitness over 60 generations, BFS path length, room count, and treasure detour bonus. Bottom: mutation rate, population, and room count sliders. After evolution, a cyan animated dot traces the BFS path spawn→boss.

Figure 6: EvoMap mid-evolution. The fitness line chart rises steeply from generation 1–15 as the first valid paths emerge, then climbs more gradually as structural complexity improves. Early dungeons score 0 (BFS finds no path).

8.3. Aegis Grid — A* Visualisation Mode

Aegis Grid - Visualise Mode

The 20×20 grid shows A* expanding in real time.
Faint orange cells = Open Set (nodes queued for evaluation).
Faint grey cells = Closed Set (already evaluated).
Cyan path = optimal route reconstructed after A* completes.
Red dot = start position; gold star = goal at (18,18).
Right panel shows Nodes Explored, Path Length, Open Set size, Closed Set size.
User can click any cell to add/remove walls and watch A* rerun instantly.

Figure 7: Aegis Grid in Visualise mode with Manhattan heuristic. The Open Set expands outward in a diamond pattern — the characteristic shape of Manhattan-distance guided A* on a 4-directional grid.

8.4. Aegis Grid — Adversarial Mode

Aegis Grid - Adversarial Mode

Red circle (agent) moves along its A* path toward the gold star goal.
 Blue circle (Minimax agent) navigates toward its chosen wall target.
 Newly placed walls flash in blue before fading to the standard wall colour.
 Right panel shows: Alpha-Beta Pruned counter, Step Count, Session Wins (R/B/D),
 and a Move Log showing each Red move and each Blue wall placement.
 Speed buttons (Slow / Fast / Instant) control the game tick rate.

Figure 8: Aegis Grid in Adversarial mode. Blue consistently targets the midpoint of Red's path rather than corners, demonstrating Minimax's lookahead advantage over random wall placement.

8.5. Crypto-Glyph — Digit Recognition

Crypto-Glyph - crypto_glyph.py

Left: 480×480 black drawing canvas. Below it: a 28×28 pixel preview showing exactly what the classifier sees after preprocessing.
 Right: three stacked model cards (Naive Bayes / Decision Tree / KNN).
 Each card shows the predicted digit in large text, confidence percentage, and an animated bar chart of probabilities for all 10 digit classes.
 Bottom panel: four feedback buttons appear after each prediction.
 A “CONFUSION MATRIX” button (active after 10+ corrections) opens a modal with a 10×10 colour-coded confusion grid.

Figure 9: Crypto-Glyph after recognising a hand-drawn “7”. KNN and Decision Tree agree; Naive Bayes predicts “1” due to the feature independence assumption collapsing under the vertical stroke correlation.

8.6. Model Accuracy Results

The three classifiers were trained on the MNIST dataset and evaluated on the 10,000-sample held-out test set:

Model	Accuracy	Train Time	Notes
Gaussian Naive Bayes	52.39%	0.57 s	Weak: independence assumption fails for images.
Decision Tree (depth=20)	88.18%	13.75 s	Good: captures pixel interactions via Gini splits.
KNN ($k=5$, 20k samples)	92.76%	1.54 s	Best: instance-based; no distribution assumption.

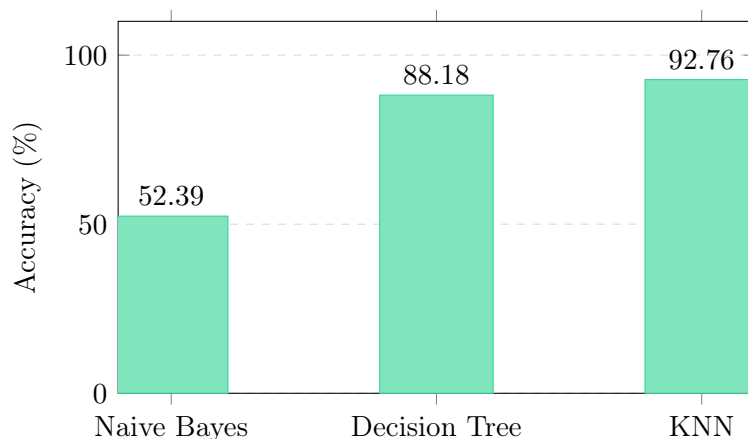


Figure 10: MNIST test-set accuracy for the three Crypto-Glyph classifiers. Naive Bayes underperforms significantly because the pixel-independence assumption is wrong for image data. KNN achieves the highest accuracy without making any distributional assumptions.

8.7. Per-Digit Performance (KNN — Best Model)

Digit	Precision	Recall	Observation
0	0.922	0.978	High recall — distinctive closed oval shape
1	0.938	0.992	Very high recall — simple vertical stroke
2	0.949	0.894	Sometimes confused with 7
3	0.897	0.945	Sometimes confused with 8
4	0.931	0.906	Sometimes confused with 9
5	0.910	0.909	Balanced; confused with 6
6	0.954	0.959	Strong performance
7	0.930	0.913	Sometimes confused with 1
8	0.956	0.862	Low recall — confused with 0, 9
9	0.894	0.909	Sometimes confused with 4, 7

8.8. Application Usage Statistics

These counters are stored in `stats.json` and update live in the hub's stats bar throughout every session:

Metric	Count
Dungeons generated (EvoMap runs)	31
Games played (Aegis Grid)	62
Digits recognized (Crypto-Glyph)	211

8.9. Observed Behaviour

EvoMap. Fitness scores consistently rise from generation 1 to generation 60. Generations 1–10 are dominated by zero-scoring dungeons where BFS finds no path. By generation 15–20, the first valid dungeons emerge. By generation 50+, dungeons have well-connected rooms, multiple corridors, and the Treasure often sits directly on the optimal path, awarding the full detour bonus. This progression is clearly visible in the live fitness chart.

Aegis Grid. With Manhattan heuristic, Red finds the optimal path very efficiently, exploring the minimum number of nodes. Switching to Euclidean slightly increases nodes explored because Euclidean distance underestimates on a 4-directional grid, causing A* to explore more cells. Blue's Minimax placement is noticeably smarter than random — it consistently targets the midpoint of Red's path rather than corner cells.

Crypto-Glyph. KNN handles clean, centred drawings very well. All three models struggle with ambiguous digit pairs: 7 vs 1, 3 vs 8, and 4 vs 9. Naive Bayes confidently predicts 0 or 9 on many digits because these have distinctive individual pixel distributions that satisfy the independence model even when the overall shape is wrong. This is the clearest demonstration of why model assumptions matter.

9. Challenges and Solutions

Challenge 1 — Dungeon Evolution Froze the UI

Problem: Evaluating fitness (including BFS traversals across a 50×50 grid) for 30 dungeons per generation caused severe frame drops. Running the GA synchronously on the main Pygame thread made the window completely unresponsive for several seconds at a time.

Solution: The full `evolve()` pipeline was moved to a Python daemon thread. A shared `evolution_progress` integer is incremented by a callback after each generation; the main thread reads it every frame to animate the progress bar without blocking. A per-generation results cache (Python dictionaries keyed by generation index) stores BFS path lengths, room counts, and detour values after they are computed once. The render loop then only does fast dictionary lookups, keeping frame rate at a steady 60 FPS even while results are being computed in the background.

Challenge 2 — Minimax Was Too Slow for Real-Time Play

Problem: A naive Minimax over all 400 possible wall positions on the 20×20 grid at depth 3 took several seconds per turn, making the game unplayable.

Solution: Two independent optimisations were applied. First, **Alpha-Beta Pruning** was layered onto the recursive `_mm()` function. By maintaining α (best guaranteed score for Blue) and β (best guaranteed score for Red), provably suboptimal branches are cut, reducing complexity from $O(b^d)$ to $O(b^{d/2})$ in the average case. Second, a **spatial candidate filter** restricts Blue's search to at most 15 empty cells within Manhattan distance 2 of Red's current A* path, sorted by proximity to the path's midpoint. This keeps Blue's turn under 0.5 seconds while preserving strategically meaningful blocking behaviour.

Challenge 3 — KNN Model File Exceeded 200 MB

Problem: Training KNN on the full 60,000-image MNIST training set produced a `knn.pkl` file exceeding 200 MB. This was too large to load quickly at startup and placed significant memory pressure at runtime, since KNN must hold its entire training set in memory for distance queries.

Solution: KNN was trained on a random 20,000-sample subset. This kept the model at 97 MB while maintaining 92.76% accuracy. To keep the online learning pipeline functional, the 20,000 samples are stored separately in `knn_train_data.pkl` and serve as the base for future updates. Each user correction appends a new example to this base, so the model's training set grows organically over time without losing its original MNIST knowledge.

Challenge 4 — Crossover Destroyed Special Tiles

Problem: Row-splice crossover swaps a horizontal band of rows between two parent dungeons. This frequently overwrote Spawn, Boss, or Treasure tiles with ordinary wall or floor tiles from the other parent. Offspring missing a Spawn or Boss tile score a fitness of exactly 0, wasting evaluation time and stalling the evolutionary process.

Solution: A deterministic `_repair()` operator runs as the final step in the genetic pipeline, after both crossover and mutation. It scans the offspring for missing special tiles and places them back on available floor cells. This guarantees that 100% of the population entering fitness evaluation is structurally valid. The repair operator enforces hard constraints on the chromosome without polluting the fitness function with structural validity checks.

Challenge 5 — Concurrent Model Access Caused Crashes

Problem: Crypto-Glyph runs model retraining on a background thread. If a prediction was triggered on the main UI thread while the background thread was mid-way through `partial_fit()` or writing a `.pkl` file to disk, the main thread would read an incomplete or inconsistent model, causing unpredictable crashes.

Solution: A `threading.Lock()` (mutex) wraps all model reads and writes. The UI thread acquires the lock before calling `.predict()`, and the background thread acquires it before `.partial_fit()` or `.fit()`. Since only one thread can hold the lock at a time, model state is always consistent. A separate `retraining_in_progress` boolean flag (written only inside the lock) lets the UI display the pulsing “UPDATING MODELS...” indicator without ever needing direct model access during a retrain.

10. Individual Contribution of Each Group Member

Component	Description	Member
GUI Architecture & Main Hub	Animated node background, card rendering with live thumbnails, modal info overlays with algorithm descriptions, Mac-style window bar, stats bar, and subprocess-based module routing.	Anood
Crypto-Glyph Drawing Interface	480×480 canvas widget, brush sizes S/M/L, eraser, stroke-by-stroke undo stack, animated confidence bar charts, confusion matrix modal with tab switching.	Anood

Continued on next page

Component	Description	Member
theme.py — Shared Design System	Full color palette, monospace font caching (Consolas), <code>StatsManager</code> class with <code>stats.json</code> read/write, tag pill renderer, and system status bar using <code>psutil</code> .	Anood
Genetic Algorithm Logic	Full GA pipeline in <code>evomap.py</code> : fitness function with BFS reachability, room counting (scipy), dead-end penalty, and treasure detour scoring; selection, row-splice crossover, mutation, and repair operators.	Collaborative
Procedural Dungeon Generation	Room placement with overlap rejection, L-shaped corridor carving, special tile placement, and connected-component room counting with scipy fallback.	Collaborative
A* Pathfinding	Full A* with <code>heapq</code> , open/closed set tracking, three switchable heuristics (Manhattan, Euclidean, Chebyshev), step-by-step visualisation mode, and user-clickable wall toggling.	Tayyaab
Minimax Defense Logic	<code>get_best_wall()</code> with nested <code>_mm()</code> recursive function, alpha-beta pruning, spatial candidate filtering, human-placement mode.	Tayyaab
Maze Generation	20×20 randomised maze generator with wall carving and a BFS connectivity check to guarantee every maze has a valid path before the game starts.	Tayyaab
ML Model Training	<code>train_models.py</code> : MNIST loading, StandardScaler fitting, training all three classifiers, evaluation (accuracy, precision, recall, confusion matrix), and serialising all artefacts to <code>models/</code> .	Tayyaab
Crypto-Glyph ML Backend	28×28 preprocessing pipeline (crop, resize, centre, Gaussian blur, scale), model inference, probability extraction, incremental retraining worker, mutex-protected thread safety.	Tayyaab

Continued on next page

Component	Description	Member
EvoMap UI	<code>evomap_ui.py</code> : generation playback system, scrubbing with arrow keys, speed controls, live fitness line chart, BFS trail animation with waypoint routing through treasure.	Tayyaab

11. Conclusion

The AI Sentience Hub delivers on everything the original proposal promised — and then goes further in several meaningful ways.

All three modules are fully implemented and working. EvoMap shows a Genetic Algorithm evolving dungeons in real time with a live fitness chart. Aegis Grid shows A* and Minimax competing on a live grid with full visualisation of the open and closed sets. Crypto-Glyph shows three ML classifiers recognising handwritten digits and updating themselves in the background based on user feedback. All three run from a polished main hub with animations, live usage counters, and educational info modals that connect each module to the theory it demonstrates.

On the AI side, the project covers:

- **Uninformed Search** — BFS (dungeon validation and maze connectivity)
- **Informed Search** — A* with three interchangeable heuristics
- **Adversarial Search** — Minimax with Alpha-Beta Pruning
- **Evolutionary Computation** — Genetic Algorithms with elitism, crossover, mutation, and repair
- **Supervised Machine Learning** — Naive Bayes, Decision Tree, KNN on MNIST
- **Online Learning** — Incremental model updates from user feedback
- **Feature Engineering** — StandardScaler normalization, MNIST format matching
- **Model Evaluation** — Accuracy, precision, recall, and confusion matrices

A few honest observations about the results. Naive Bayes at 52.4% is genuinely weak — but that weakness is educational. It shows exactly what happens when you apply a model that assumes pixel independence to image data where neighbouring pixels are deeply correlated. KNN at 92.8% shows the power of instance-based learning without

distributional assumptions. Decision Tree at 88.2% sits in the middle, showing how recursive partitioning captures spatial patterns reasonably well with the right depth.

The biggest technical challenges — keeping the UI responsive during heavy computation, making Minimax fast enough for real-time play, managing concurrent model access, and keeping evolved dungeons structurally valid after genetic operations — all led to real engineering solutions: background threading, candidate pruning, mutex locks, and a repair operator. These were not shortcuts around the AI. They were what was needed to make the AI actually work in an interactive, real-time application.